```
// Stimulus Generation
initial begin
   fork
      clock_gen();
      monitor_DUT();
   join_none

   assert_reset;

   repeat(10000) begin
      assert(c_txn.randomize());
      fork
         drive_stimulus(c_txn);
         golden_model(c_txn);
         check_result(c_txn);
      join
   end

   $display("%t: At end of test e
   $stop;
end

task clock_gen();
   forever #1 clk = ~clk;
endtask
```

```
task monitor_DUT();
   forever begin
      @(negedge clk);
      if (!rst_n || !load_n || ce);
         $display("time = %0t: rst_n = %0b, load_n = %0b
   end
endtask

task drive_stimulus(input counter_transaction c_txn_c);
   rst_n = c_txn.rst_n;
   load_n = c_txn.load_n;
   up_down = c_txn.up_down;
   ce = c_txn.ce;
   data_load = c_txn.data_load;
endtask
```

Karnavaen

```
module tb;
bit clk , rst_n, request, ack;

receiver r1(clk ,rst_n, request, ack);

initial
   forever #1 clk = ~clk;

initial begin
   // Reset
   rst_n = 0;
   @(negedge clk);
   rst_n = 1;
   // Drive data
   request = 1;
   // Check if ack is received within maximum delay
   fork
      begin
         wait (ack == 1);
      end
      begin
         repeat (5) @(negedge clk);
         $display("Ack timed out",);
         $stop;
      end
   join_any
   disable fork;
end
endmodule
```

```
assert_C: assert property(@(posedge clk) disable iff (reset) C == $past(A) + $past(B));
cover_C: cover property(@(posedge clk) disable iff (reset) C == $past(A) + $past(B));

always_comb begin
   if(reset)
    assert_r: assert final(C == 0);
 end
```

```
1- When signal_a rises, then starting next cycle, signal_b eventually shall fall
 >> assert property (@(posedge clk) ($rose(signal_a) |=> $fell(signal_b) [->1]));

2- When valid signal is high, then after 1 clk cycle, wr_ack should remain high until done is high
 >> assert property (@(posedge clk) (valid |=> (wr_ack throughout (done [->1]) )));

3- When signal req rises, then after 1 clk cycle it is expected the ack eventually be high ,and it's expected to get low the following clk cycle
 >> assert property (@(posedge clk) ($rose(req) |=> ack[->1] ##1 ~ack));
```

```
interface vending_machine_if(clk);
  parameter WAIT = 2'b00;
  parameter Q_25 = 2'b01;
  parameter Q_50 = 2'b11;
  input clk;
  logic Q_in, D_in, rstn, dispense, change;

  modport DUT (input Q_in, D_in, rstn, clk, output dispense, change);

  modport TEST (output Q_in, D_in, rstn, input clk, dispense, change);

  modport MONITOR (input Q_in, D_in, rstn, clk, dispense, change);

endinterface
```

Karim Waseem

```
module vending_machine_monitor(vending_machine_if.MONITOR v_if);

//Test monitor and results
initial begin
 $monitor("rstn = %b, clk = %b, Q_in = %b, D_in = %b, dispense = 
end

endmodule
```

```
module vending_machine_sva(vending_machine_if.DUT v_if);

property p_dollar;
 @(posedge v_if.clk) v_if.D_in |-> (v_if.dispense && v_if.change);
endproperty

property p_quarter_dispense;
 @(posedge v_if.clk) $rose(v_if.Q_in) |-> ##2 v_if.dispense;
endproperty

property p_quarter_v_if_change;
 @(posedge v_if.clk) (v_if.Q_in |-> !(v_if.change));
endproperty

Dollar_assertion: assert property(p_dollar);
Quarter_v_if_dispense_assertion: assert property(p_quarter_dispense);
Quarter_v_if_change_assertion: assert property(p_quarter_v_if_change);

Dollar_coverage: cover property(p_dollar);
Quarter_v_if_dispense_coverage: cover property(p_quarter_dispense);
Quarter_v_if_change_coverage: cover property(p_quarter_v_if_change);

endmodule
```

```
module vending_machine_tb(vending_machine_if.TEST v_if);
```

```
module vending_machine_top();
  bit clk;

  //clock generation
  initial begin
    clk = 0;
    forever
      #1 clk = ~clk;
  end

  vending_machine_if v_if(clk);
  vending_machine_tb tb(v_if);
  vending_machine dut(v_if);
  vending_machine_monitor mon(v_if);
  bind vending_machine vending_machine_sva vending_machine_sva_inst(v_if);

endmodule
```

```
module vending_machine(vending_machine_if.DUT v_if);
```

Kareem Waseem